SMART MOVIE RECOMMENDATION SYSTEM

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Artificial Intelligence and Machine Learning

Submitted by

D. Swetha **(24B11AI087)**

K. Rishi Sahithi **(24B11AI154)**

K. Keerthi Reddy **(24B11AI159)**

Under the esteemed guidance of

Mr. G. Siva Sankar M.Tech ,Ph.D.

Assistant Professor, Department of AI & ML



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

Surampalem, Kakinada District, Andhra Pradesh – 533437

2026 – 2027

ADITYA UNIVERSITY

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the project work entitled **"Smart Movie Recommendation System"** is a bonafide record of the work carried out by

D. Swetha	24B11AI087
K. Rishi Sahithi	24B11AI154
K. Keerthi Reddy	24B11AI159

in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Artificial Intelligence and Machine Learning** of Aditya University, Surampalem, during the academic year 2026 – 2027. The work embodied in this report has not been submitted to any other university or institution for the award of any degree or diploma.

Project Guide
Mr. G. Siva Sankar M.Tech ,Ph.D.
Assistant Professor
Department of AI & ML

Head of the Department
Dr. K. Naga Bhargavi M.Tech ,Ph.D.
Professor & Head
Department of AI & ML

External Examiner

DECLARATION

We hereby declare that the project report entitled **"Smart Movie Recommendation System"** is a record of original work done by us under the guidance of **Mr. G. Siva Sankar**, Assistant Professor, Department of AI & ML, Aditya University, and is submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Artificial Intelligence and Machine Learning**. The results embodied in this report have not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma. All sources of information and datasets used have been acknowledged in the references.

Place: Surampalem

Date: _____

Project Associates

D. Swetha (24B11AI087)
K. Rishi Sahithi (24B11AI154)
K. Keerthi Reddy (24B11AI159)

ACKNOWLEDGEMENT

It is with immense pleasure that we would like to express my indebted gratitude to my project guide **Mr. G Siva Sankar**, Department of Artificial Intelligence and Machine Learning, who has guided us a lot and encouraged in every step of the project work. His valuable moral support and guidance throughout the project helped me to a greater extent.

We extend our sincere gratitude to **Dr. B. Manikyala Rao** (Dy.HOD) for his continuous inspiration and for arranging the essential facilities and resources required for our project.

We are grateful to **Dr. K. Naga Bhargavi (HOD),** for inspiring me all the way and for arranging all the facilities and resources needed for our project.

We wish to thank our **Dr. M.V. Rajesh**, Associate Dean (School of Computing), **Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics)** and **Dr. G. Suresh, Registrar**, for their encouragement and support during the course of my project.

We would like to extend my sincere thanks to **Dr. M. B. Srinivas, Vice- Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor and Management, Aditya University** for their unconditional support in providing me with the best infrastructural facilities and state-of-the-art laboratories during my project.

ABSTRACT

The Smart Movie Recommendation System is a machine learning-based web application designed to help users discover movies based on their interests, moods, and preferences. It addresses the difficulty of selecting suitable movies by considering content similarity, preferred language, minimum rating, and runtime. The system uses a dataset containing 4,803 movie records, including genres, keywords, overviews, cast, and director information. It applies TF-IDF vectorization and cosine similarity to identify movies with similar content. The application also integrates the TMDB API to search for movies and retrieve additional recommendations. A Streamlit interface allows users to select their preferences and receive ranked movie suggestions. A weighted scoring method combines content-based candidate ordering with mood, language, runtime, and rating scores. The system also provides brief explanations for recommendations based on available movie information and user preferences. Developed using Python, Pandas, scikit-learn, Streamlit, Requests, and Pickle, the application demonstrates a practical approach to personalized movie discovery. Its current mood analysis is rule-based, and recommendation availability depends on dataset coverage and TMDB API access.

Keywords: Machine Learning, Movie Recommendation, Content-Based Filtering, TF-IDF, Cosine Similarity, Streamlit, TMDB API, Context-Aware Recommendation, Python.

TABLE OF CONTENTS

Title **Page No.**

CERTIFICATE ..... **i**

DECLARATION ..... **ii**

ACKNOWLEDGEMENT ..... **iii**

ABSTRACT ..... **iv**

LIST OF FIGURES ..... **vii**

LIST OF TABLES ..... **viii**

LIST OF ABBREVIATIONS ..... **ix**

CHAPTER 1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Objectives of the Project 2

1.4 Scope of the Project 2

1.5 Organization of the Report 3

CHAPTER 2 SYSTEM OVERVIEW **4**

2.1 Existing System 4

2.2 Limitations of Existing System 4

2.3 Proposed System 5

2.4 Software Requirements 5

2.5 Hardware Requirements 6

2.6 Feasibility Study 6

CHAPTER 3 SYSTEM IMPLEMENTATION **7**

3.1 Overview of Development Tools 7

3.2 Dataset Description 7

3.3 Deployment 8

CHAPTER 4 SYSTEM ARCHITECTURE **9**

4.1 Overall Architecture 9

4.2 Machine Learning Workflow 9

4.3 Request Processing Flow 10

4.4 Modules of the System 10

4.5 Privacy and Security Design 11

CHAPTER 5 DATA PREPROCESSING **12**

5.1 Introduction 12

5.2 Data Cleaning 12

5.3 Feature Selection and Tag Creation 12

5.4 Missing values and Duplicates 13

5.5 TF-IDF Feature Extraction 13

5.6 Data Representation....13
5.7 Saved Artifacts....14
5.8 Summary....14
CHAPTER 6 METHODOLOGY AND ALGORITHMS....15
6.1 Introduction....15
6.2 Text Representation: TF-IDF....15
6.3 Cosine Similarity....16
6.4 Content Based Filtering....15
6.5 TMDB API Integration....17
6.6 User Preference Based Ranking....17
6.7 Rule Based Recommendation Explanation....18
6.8 Summary....18
CHAPTER 7 RESULTS AND ANALYSIS....19
7.1 Introduction....19
7.2 Experimental Setup....19
7.3 Recommendation Results....19
7.4 User Preference Results....20
7.5 Comparison of Recommendation Techniques....21
7.6 Key Findings....21
7.7 Summary....21
CHAPTER 8 SYSTEM TESTING....22
8.1 Testing Strategy....22
8.2 Functional Testing....22
8.3 Sample Test Cases....22
8.4 Validation Summary....23
CHAPTER 9 OUTPUT SCREENS....24
CHAPTER 10 CONCLUSION AND FUTURE SCOPE....28
10.1 Conclusion....28
10.2 Future Scope....28
CHAPTER 11 REFERENCES....30
APPENDIX A: API ENDPOINTS....31

LIST OF FIGURES

Figure **Page No.**

Fig 4.1: Overall System Architecture.....9
Fig 4.2: End-to-end machine learning workflow 10
Fig 5.1: Text Preprocessing and Tag Creation.....12
Fig 7.1: Movie Recommendation System Output 20
Fig 7.2:Recommendation based on user preferences.....20
Fig 9.1: Home Screen of the Smart Movie Recommendation.....24
Fig 9.2: Movie Search and Selection 24
Fig 9.3: User Preference Selection.....25
Fig 9.4: Movie Recommendation.....25
Fig 9.5: Recommendation Explanation.....26
Fig 9.6: Final Recommendation.....26,27

LIST OF TABLES

Table **Page No.**

Table 2.1: Software requirements 5
Table 2.2: Hardware requirements 6
Table 3.1: Datasets used in the project 8
Table 4.1: Modules of the system.....10
Table 7.1: Comparison of recommendation techniques.....21
Table 8.1: Sample test cases 22
Table A.1: REST API endpoints of the backend 31

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
ML	Machine Learning
NLP	Natural Language Processing
TF-IDF	Term Frequency–Inverse Document Frequency
TMDB	The Movie Database
UI	User Interface
URL	Uniform Resource Locator
JSON	JavaScript Object Notation

CHAPTER 1
INTRODUCTION

1.1 Introduction

Movies are a popular form of entertainment, but choosing a suitable movie can take time because online catalogues contain thousands of titles across many genres and languages. A viewer may want a movie similar to one they enjoyed, or may be looking for a particular mood such as comedy, romance, suspense or a thought-provoking story. Browsing manually through large catalogues does not always make it easy to find relevant choices.

A movie recommendation system helps users discover titles by comparing movie information and identifying items that share meaningful characteristics. Content-based filtering is one approach: it recommends items that are similar to the selected item based on its descriptive information. In this project, movie genres, keywords, plot overview, cast and director are combined into a text field called tags. TF-IDF converts these descriptions into numerical vectors, and cosine similarity is used to find related movies.

The Smart Movie Recommendation System combines this local content-based approach with the TMDB API, which supports movie search and can provide additional similar-title candidates. A Streamlit interface allows users to enter a movie title and select preferences such as mood, language, minimum rating and runtime category. The application ranks available candidates and gives a short explanation for each recommendation.

1.2 Problem Statement

Users often face difficulty selecting a movie from a large collection because searching by title or browsing by genre alone does not account for similarities in plot, keywords, cast and director. General lists may not reflect a user's current mood, language preference, preferred duration or rating threshold. The project aims to develop a simple system that uses movie metadata and similarity techniques to recommend related movies while allowing users to express these preferences.

1.3 Objectives of the Project

To prepare a usable movie dataset by combining and cleaning movie and credits information.

To represent movie descriptions using TF-IDF features.

To calculate content similarity between movies using cosine similarity.

To integrate TMDB search and similar-movie results as an additional source of candidates.

To consider mood, language, minimum rating and runtime preferences when ranking candidates.

To display up to five available recommendations through a simple Streamlit interface.

To provide short, understandable explanations for why a movie was recommended.

1.4 Scope of the Project

The project focuses on content-based recommendations using metadata from the TMDB 5000 Movies and Credits dataset, which provides 4,803 usable movie records in the prepared dataset. The local recommendation model uses genres, keywords, overview, cast and director information. TMDB integration helps search for a requested title and retrieve further similar-movie candidates. The user can choose mood, language, rating and runtime preferences.

The system is a prototype and does not use a trained collaborative-filtering model or a personal viewing-history database. Mood scoring is based on predefined rules, and external metadata can be incomplete. Recommendations depend on the local dataset, network access and the API response.

1.5 Organization of the Report

Chapter 1 introduces the project, its problem, objectives and scope. Chapter 2 discusses existing approaches, their limitations and the proposed system. Chapter 3 describes development tools, the dataset and external services. Chapter 4 explains the architecture and modules. Chapter 5 describes data preparation and feature extraction. Chapter 6 presents the recommendation methods and ranking logic. Chapter 7 discusses the workflow, outputs and findings. Chapter 8 describes testing. Chapter 9 is reserved for application screenshots. Chapter 10 concludes the report and presents future improvements. Chapter 11 lists references, followed by an appendix of tools and saved artifacts.

CHAPTER 2
SYSTEM OVERVIEW

2.1 Existing System

Common movie discovery platforms provide search bars, genre categories, popularity lists, ratings and recommendations generated from their own catalogues. Basic catalogue search returns movies whose titles match the query, while genre filtering groups movies into broad categories. Some services use user behaviour and viewing history to personalize results.

A simple academic recommendation prototype can also compare movie descriptions using text similarity. Such systems are useful when user-rating history is unavailable, because they can recommend titles based on the selected movie's content.

2.2 Limitations of Existing System

Manual browsing across large catalogues can take time.

Title search alone does not identify movies with similar plot themes or creative details.

Genre-only recommendations can be broad and may miss similarities in overview, keywords, cast or director.

Some recommendation methods require user histories or rating data that may not be available for a new user.

External movie information and recommendation services depend on network availability and may return incomplete metadata.

Recommendations may not explain which features or preferences contributed to a result.

2.3 Proposed System

The proposed Smart Movie Recommendation System combines local content-based filtering with external movie search. The dataset is preprocessed into a single tags field containing overview, genres, keywords, cast and director. A TF-IDF vectorizer converts tags into a sparse feature matrix. When a selected movie is available locally, cosine similarity is used to order similar local movies.

The app also searches TMDB and requests similar movies when the service is available. Local and external candidates are combined and duplicates are removed. Preference scores for mood, language, runtime and rating are then used in a weighted ranking method. Streamlit displays the available recommendations and a short "Why this movie?" explanation.

2.4 Software Requirements

Python	Main programming language for preprocessing and recommendation logic.
Pandas	Loading, joining, cleaning and manipulating movie data.
scikit-learn	TF-IDF vectorization and cosine similarity.
Streamlit	Web interface for user inputs and recommendation results.
Requests	HTTP requests to the TMDB API.
Pickle	Saving and loading prepared data, vectorizer and vectors.
Jupyter Notebook	Exploration, preprocessing and generation of saved artifacts.
Web browser	Accessing the Streamlit application.

2.5 Hardware Requirements

Component	Minimum / Recommended Requirement
Processor	Modern dual-core processor or better.
Memory	4 GB RAM minimum; 8 GB or more recommended for comfortable development.
Storage	Enough free space for datasets, notebook, application and saved artifacts.
Network	Internet access for TMDB search and similar-movie requests.
Display	Standard laptop or desktop display for the Streamlit interface.

2.6 Feasibility Study

Technical feasibility: The system uses established Python libraries and a publicly available movie dataset. TF-IDF and cosine similarity are supported by scikit-learn, while Streamlit provides a straightforward interface.

Economic feasibility: The core tools are open-source and can be used without purchasing commercial software. TMDB API use is subject to its terms, access rules and rate limits.

Operational feasibility: The interface requires only a movie title and preference selections, so it is suitable for a demonstration by students and general users.

Schedule feasibility: The project can be developed in stages: data preparation, similarity calculation, interface development, API integration, testing and documentation.

CHAPTER 3
SYSTEM IMPLEMENTATION

3.1 Overview of Development Tools

The Smart Movie Recommendation System is developed using Python and various libraries for data processing, recommendation generation, and web application development. The main tools used are Jupyter Notebook, Scikit-learn, Streamlit, and the TMDB API.

3.1.1 Python and Jupyter Notebook

Python is used as the main programming language for the project. Jupyter Notebook is used to load the dataset, preprocess movie information, and implement the recommendation algorithm.

3.1.2 Pandas and NumPy

Pandas is used to load, organize, and process movie data. NumPy supports numerical operations required during data processing and recommendation calculations.

3.1.3 Scikit-learn

Scikit-learn is used to implement TF-IDF vectorization and cosine similarity. TF-IDF converts movie descriptions and related information into numerical vectors, while cosine similarity identifies movies with similar content.

3.1.4 Streamlit

Streamlit is used to develop the user interface. It allows users to enter a movie title, select preferences such as mood and language, and view recommended movies with explanations.

3.1.5 Requests Library and TMDB API

The Requests library is used to communicate with The Movie Database (TMDB) API. The API helps search for movies and retrieve additional recommendations.

3.2 Dataset Description

The project uses the TMDB 5000 Movies and Credits datasets. After preprocessing and combining the required information, the dataset contains 4,803 movies.

Important features include movie title, genres, keywords, overview, cast, and director. These features are combined into a single tags column, which is used to identify similarities between movies.

Feature	Description
Title	Name of the movie
Genres	Categories of the movie
Keywords	Important terms related to the movie
Overview	Short description of the movie
Cast	Actors in the movie
Director	Director of the movie
Tags	Combined movie information used for recommendations

The TF-IDF technique converts the movie tags into numerical vectors with 5,000 features. Cosine similarity is then used to measure the similarity between movies and identify suitable recommendations.

3.3 Deployment

The application is run locally using Streamlit. When a user enters a movie title, the system searches for the movie using the TMDB API and generates recommendations based on content similarity and user preferences.

The system considers factors such as mood, language, movie rating, and runtime while ranking recommendations. It displays up to five recommended movies along with brief explanations for the recommendations.

The TMDB API access token is stored separately in Streamlit's secrets configuration to avoid exposing sensitive credentials. The application is tested to ensure that movie searches, recommendations, preference selection, and result display work correctly.

CHAPTER 4
SYSTEM ARCHITECTURE

4.1 Overall Architecture

The system has four main parts: the user interface, recommendation logic, local data artifacts and the external TMDB service. The interface collects a title and preferences. Recommendation logic creates candidates, scores them and prepares explanations. Local pickle artifacts provide prepared movie information and TF-IDF representations. TMDB supplies external search and similar-title candidates.

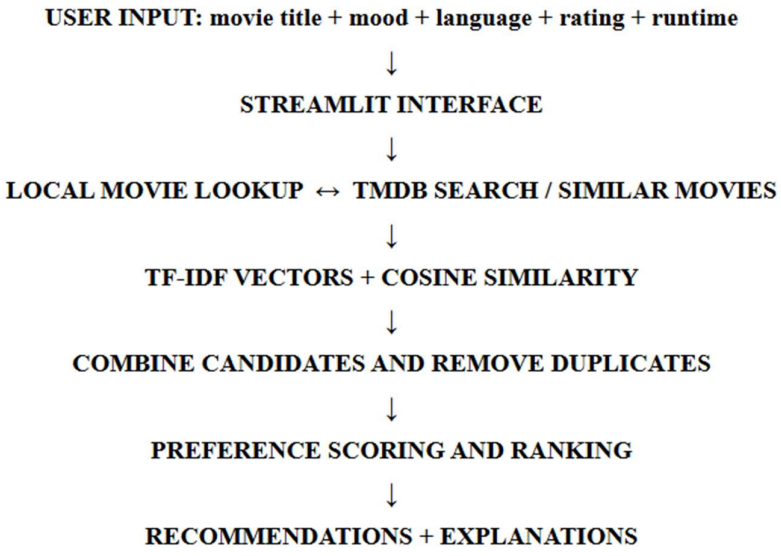


Fig. 4.1: Overall system architecture

4.2 Machine Learning Workflow

The machine learning part of the Smart Movie Recommendation System follows the end-to-end workflow shown in Fig. 4.2. Movie data is collected from the TMDB 5000 Movies and Credits datasets, cleaned, and preprocessed. Important features such as genres, keywords, overview, cast, and director are combined into a single tags column. TF-IDF converts the text into numerical feature vectors, and cosine similarity is used to identify movies with similar content. When a user selects a movie, the system combines these similarity results with recommendations from the TMDB API and ranks

them according to user preferences. Finally, the application displays up to five recommended movies with explanations.



Fig 4.2: End-to-end machine learning workflow

4.3 Request Processing Flow

The process begins when the user submits the form. The application checks the local dataset and queries TMDB. Local recommendations are ordered using cosine similarity. External candidates are added where available. The application evaluates candidates against selected preferences, builds the final ranking and renders the results. If a title is not found or an external request fails, the application should display an appropriate message or continue with candidates it can obtain.

4.4 Modules of the System

Module	Description
1. Data Collection and Preprocessing	Loads movie and credits datasets, joins records, cleans selected fields and creates tags.
2. Content-Based Recommendation	Uses TF-IDF vectors and cosine similarity to identify locally available movies with related content.

3. TMDB Integration	Searches the external catalogue and retrieves similar-movie candidates.
4. Context and Preference Analysis	Uses predefined mood rules and user choices for language, rating and runtime category.
5. Ranking and Explanation	Combines candidate ordering and preference scores, ranks movies and generates short reasons.
6. User Interface	Uses Streamlit widgets to collect inputs and display recommendations.

4.5 Privacy and Security Design

- API Token Security: The TMDB API access token is stored in Streamlit secrets rather than directly in the application code.

- Secure API Communication: The Requests library is used to communicate with the TMDB API and retrieve movie information and recommendations.

- User Input Handling: The application checks the entered movie title and selected preferences before generating recommendations.

- Data Privacy: The system uses movie-related information and user-selected preferences to generate recommendations. It does not require users to provide personal information.

- Error Handling: API errors and unavailable movie results are handled to help the application respond appropriately without exposing sensitive credentials.

.

CHAPTER 5
DATA PREPROCESSING

5.1 Introduction

Preprocessing converts raw movie metadata into a consistent representation that can be compared by the recommendation algorithm. The project selects useful fields and combines them into a single text field called tags.

5.2 Data Cleaning

The movie and credits CSV files are loaded using Pandas. The tables are joined using the movie identifier. Relevant columns are selected for recommendation. Structured values such as genres, keywords and cast are parsed into readable names, and director information is extracted from crew data. Missing text values are handled so that the tags-building step can operate consistently.

5.3 Feature Selection and Tag Creation

The selected text features are genres, keywords, overview, cast and director. Genres describe the broad category, keywords capture themes, overview describes the plot, cast identifies performers, and director contributes creative context. These fields are combined into the tags column.

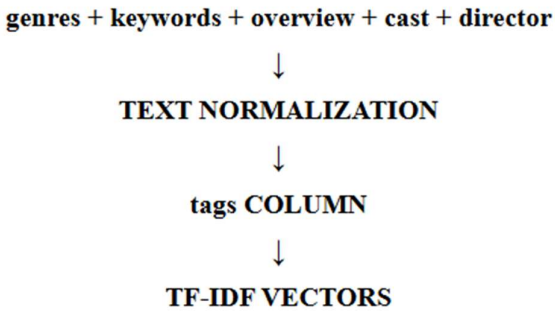


Fig. 5.1: Text preprocessing and tag creation

For example, a movie about space exploration may have tags containing terms such as “space”, “science fiction”, “survival” and “astronaut”, along with cast and director names. The vectorizer uses these words as part of the movie's text representation.

5.4 Missing Values and Duplicates

Missing values are checked during preparation because an empty overview or missing metadata can reduce the information available for similarity. Text fields used to build tags should be converted to safe strings or empty values before concatenation. Duplicate records and repeated titles should be reviewed to avoid repeated recommendations. The final working dataset contains 4,803 rows and 12 columns.

5.5 TF-IDF Feature Extraction

TfidfVectorizer is fitted on the tags column with a maximum vocabulary size of 5,000 features and English stop words removed. Common words such as “the” and “and” often do not distinguish one movie from another. The vectorizer learns a vocabulary and transforms each movie's tags into a sparse numerical vector.

The resulting vector matrix has shape (4803, 5000): 4,803 rows represent movies, and up to 5,000 columns represent vocabulary features. These vectors are used to calculate similarity when a user chooses a title.

5.6 Data Representation

Each movie is represented by a TF-IDF vector. The vector contains weighted values for words in the movie's combined tags. Movies that share meaningful terms can have vectors pointing in similar directions, which allows cosine similarity to identify related content.

5.7 Saved Feature Artifacts

The fitted vectorizer and transformed vectors are saved with Pickle so that the application can load them without repeating all preprocessing steps each time it starts. The prepared DataFrame is saved as movies.pkl. The notebook also saves a full similarity matrix, but the Streamlit app uses the vectors to calculate the selected movie's similarity when needed.

5.8 Summary

The preprocessing stage combines descriptive fields, handles missing text and creates a tags column. TF-IDF transforms the tags into a sparse matrix with 4,803 rows and 5,000 features. The saved artifacts form the input to the recommendation stage.

CHAPTER 6
METHODOLOGY AND ALGORITHMS

6.1 Introduction

The Smart Movie Recommendation System uses content-based filtering to recommend movies similar to the movie selected by the user. The system uses the TMDB 5000 Movies and Credits datasets to obtain movie information. Important features such as genres, keywords, overview, cast, and director are combined and processed using TF-IDF vectorization. Cosine similarity is then used to identify movies with similar content. The system also integrates the TMDB API and user preferences to provide more relevant recommendations.

6.2 Text Representation: TF-IDF

Term Frequency–Inverse Document Frequency (TF-IDF) is a technique used to convert text into numerical vectors. In this project, movie information is combined into a single column called tags. TF-IDF assigns weights to words based on their frequency in a movie description and their importance across the dataset.

The TF-IDF formula is:

$$\boldsymbol{TFIDF(t,d) = TF(t,d) \times IDF(t)}$$

Here, t represents a term and d represents a movie document. This technique helps the system give more importance to meaningful words and reduces the influence of common words.

Application in the project: TF-IDF is configured with a maximum of 5,000 features and English stop-word removal. The resulting vectors are used by the recommendation engine to compare movie content.

6.3 Cosine Similarity

Cosine similarity measures how similar two numerical vectors are by calculating the cosine of the angle between them.

$$\textbf{Similarity}(\boldsymbol{A}, \boldsymbol{B}) = \frac{\boldsymbol{A} \cdot \boldsymbol{B}}{\|\boldsymbol{A}\|\|\boldsymbol{B}\|}$$

The similarity value generally ranges from 0 to 1 for the non-negative TF-IDF vectors used in this project. A higher value indicates that two movies have more similar content.

Application in the project: The system compares the selected movie's vector with other movie vectors and ranks local candidates according to their similarity. Movies with higher similarity are considered more relevant recommendations.

6.4 Content-Based Filtering

Content-based filtering recommends items based on their features rather than relying on ratings from other users.

Working:

1. Collect movie information from the dataset.
2. Combine genres, keywords, overview, cast, and director into the tags column.
3. Convert the tags into numerical vectors using TF-IDF.
4. Calculate cosine similarity between movie vectors.
5. Identify and rank movies similar to the selected movie.

Advantages: It provides recommendations based on movie content, does not require other users' ratings, and can suggest similar movies to users with different interests.

Application in the project: This is the main recommendation method used to identify movies similar to the user's selected movie.

6.5 TMDB API Integration

The TMDB API provides access to movie search results and additional recommendations.

Working:

- The user enters a movie title in the application.
- The system searches for the movie using the TMDB search API.
- The TMDB similar-movies API retrieves additional candidate movies.
- These results are combined with locally generated recommendations.
- Duplicate movies are removed before the final ranking process.

Application in the project: API integration expands the available recommendation results beyond the local dataset and provides additional movie suggestions.

6.6 User Preference-Based Ranking

The system considers user preferences to make recommendations more relevant. Users can select a mood, language, minimum rating, and runtime category.

The ranking process combines content relevance with preference-based scores using the following formula:

$$S = 0.55C + 0.20M + 0.10L + 0.10R + 0.05V$$

Where:

- C = content relevance score
- M = mood score
- L = language score

- R = runtime score
- V = rating score

The system uses these weighted scores to rank the available candidates and displays up to five recommendations when enough suitable results are available.

6.7 Rule-Based Recommendation Explanations

The system provides short explanations to help users understand why a movie was recommended. These explanations are generated using predefined rules based on movie information and selected preferences.

For example, a recommendation may be explained by its similar genre, matching mood, preferred language, or suitable runtime. These explanations make the results easier to understand.

6.8 Summary

The project combines TF-IDF vectorization, cosine similarity, content-based filtering, TMDB API integration, and preference-based ranking. TF-IDF and cosine similarity identify movies with similar content, while the API provides additional candidates. Finally, user preferences help rank the recommendations and generate understandable results through the Streamlit interface.

CHAPTER 7
RESULTS AND ANALYSIS

7.1 Introduction

This chapter presents the results of the Smart Movie Recommendation System and discusses the performance of the recommendation techniques used. It explains how TF-IDF vectorization and cosine similarity help identify similar movies. It also discusses the role of TMDB API integration and user preferences in generating relevant recommendations.

7.2 Experimental Setup

The system uses the TMDB 5000 Movies and Credits datasets. After preprocessing, the dataset contains 4,803 movies. The movie information is combined into a tags column and converted into numerical vectors using TF-IDF with a maximum of 5,000 features.

Cosine similarity is used to identify movies with similar content. The application is tested with different movie titles and user preferences, including mood, language, minimum rating, and runtime. The generated recommendations are checked to determine whether they are relevant to the selected movie and preferences.

7.3 Recommendation Results

The system generates movie recommendations by comparing the content of the selected movie with other movies in the dataset. Movies with higher cosine similarity are considered more relevant.

The TMDB API provides additional similar-movie results, which are combined with local recommendations. Duplicate results are removed, and the available candidates are ranked according to content relevance and user preferences.

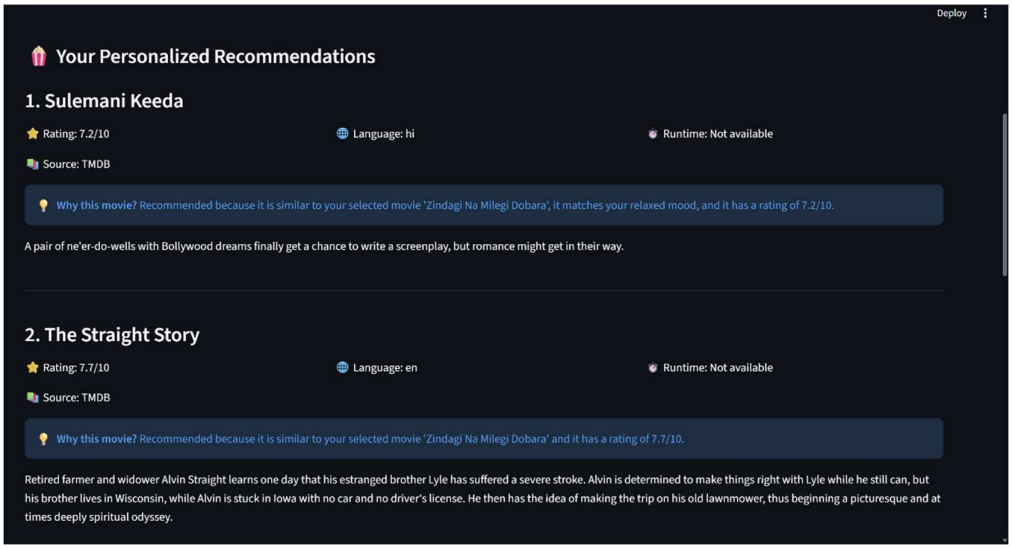


Fig. 7.1: Movie Recommendation System Output

7.4 User Preference Results

The system allows users to select preferences such as mood, language, minimum rating, and runtime category. These preferences influence the ranking of the recommended movies.

The system also provides short explanations for recommendations based on movie details and selected preferences. This helps users understand why a movie appears in the results and makes the recommendation process more transparent.

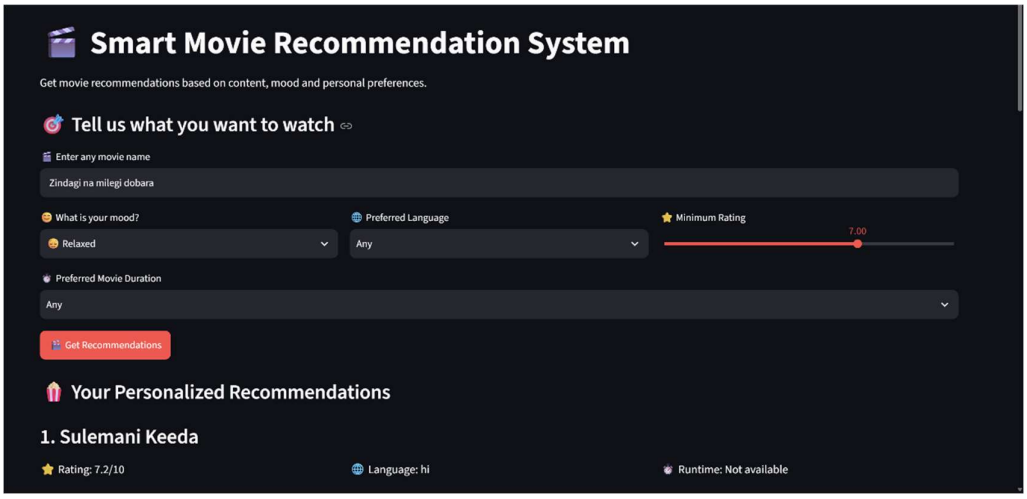


Fig. 7.2: Recommendations Based on User Preferences

7.5 Comparison of Recommendation Techniques

The project combines content-based filtering with recommendations obtained through the TMDB API.

Technique	Purpose
TF-IDF	Converts movie text into numerical vectors
Cosine Similarity	Identifies movies with similar content
TMDB API	Retrieves additional movie recommendations
Preference-Based Ranking	Ranks candidates according to user preferences

Content-based filtering provides recommendations based on movie information, while TMDB API integration expands the available candidates. Preference-based ranking helps prioritize movies that better match the user's selections.

7.6 Key Findings

- TF-IDF successfully converts movie descriptions and related information into numerical representations.
- Cosine similarity helps identify movies with similar genres, keywords, cast, director, and overview.
- TMDB API integration provides additional movie recommendations beyond the local dataset.
- User preferences help improve the relevance of the displayed results.
- Recommendation explanations make the system easier to understand.

7.7 Summary

The results demonstrate that the Smart Movie Recommendation System combines content-based filtering, API integration, and user preferences to generate movie suggestions. The system provides a simple interface and displays up to five recommendations when enough suitable candidates are available. Its effectiveness depends on the quality of movie information, API responses, and the relevance of the selected preferences.

CHAPTER 8
SYSTEM TESTING

8.1 Testing Strategy

The Smart Movie Recommendation System was tested to verify the correctness of its recommendation process, user preferences, and API integration. Testing was performed at different levels to ensure that the application works as expected.

- Unit Testing: Checks individual functions such as text preprocessing, TF-IDF vectorization, and cosine similarity calculations.
- Integration Testing: Verifies the interaction between the Streamlit application, recommendation engine, and TMDB API.
- Recommendation Testing: Checks whether the system generates relevant movie suggestions for different movie titles.
- User Interface Testing: Verifies that movie search, preference selection, and recommendation results are displayed correctly.

8.2 Functional Testing

Functional testing checks whether the main features of the application work correctly. The system was tested with different movie titles and combinations of user preferences to check the generated recommendations.

Testing also covered API responses, duplicate removal, and the display of explanations. Invalid movie searches and unavailable API results were checked to ensure that the application handles such situations appropriately.

8.3 Sample Test Cases

Table 8.1: Sample Test Cases

TestID	Test Case	Expected Result	Status
TC01	Enter a valid movie title	Movie is found and recommendations are displayed	Pass

TC02	Enter an invalid movie title	Appropriate message is displayed	Pass
TC03	Select a mood	Recommendations are ranked using the selected mood	Pass
TC04	Change the preferred language	Language preference affects ranking	Pass
TC05	Set a minimum rating	Rating preference is considered during ranking	Pass
TC06	Select a runtime category	Runtime preference is considered when data is available	Pass
TC07	Generate recommendations	Similar movies are identified using content similarity	Pass
TC08	Check recommendation results	Duplicate movies are removed	Pass
TC09	Retrieve similar movies from TMDB	Additional movie candidates are retrieved when available	Pass
TC10	Display recommendation explanations	A reason is shown for each recommendation	Pass

8.4 Validation Summary

The system's validation focuses on checking the recommendation process, user preference handling, and integration with the TMDB API. TF-IDF and cosine similarity are used to identify movies with similar content, while preference-based ranking helps prioritize relevant results.

The application is designed to display up to five recommended movies with explanations. The number and relevance of results may depend on the available dataset, API responses, and selected preferences. Testing these features helps verify the overall functionality and usability of the Smart Movie Recommendation System.

CHAPTER 9
OUTPUT SCREENS

This chapter presents the main screens of the Smart Movie Recommendation System developed using Streamlit. The screenshots demonstrate the user interface, movie search process, preference selection, and recommendation results. They show how users can select a movie, customize their preferences, and receive movie suggestions with explanations.

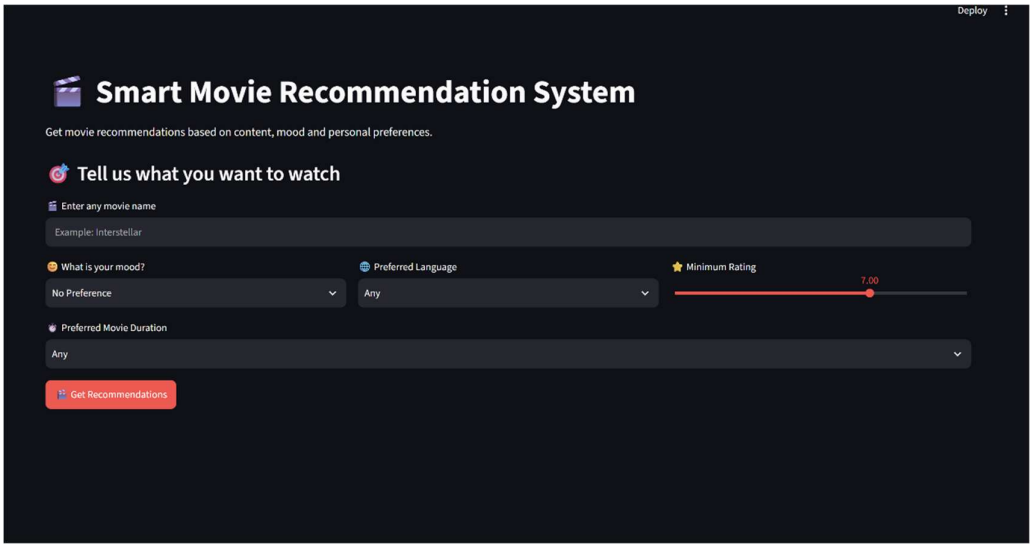


Fig. 9.1: Home Screen of the Smart Movie Recommendation System

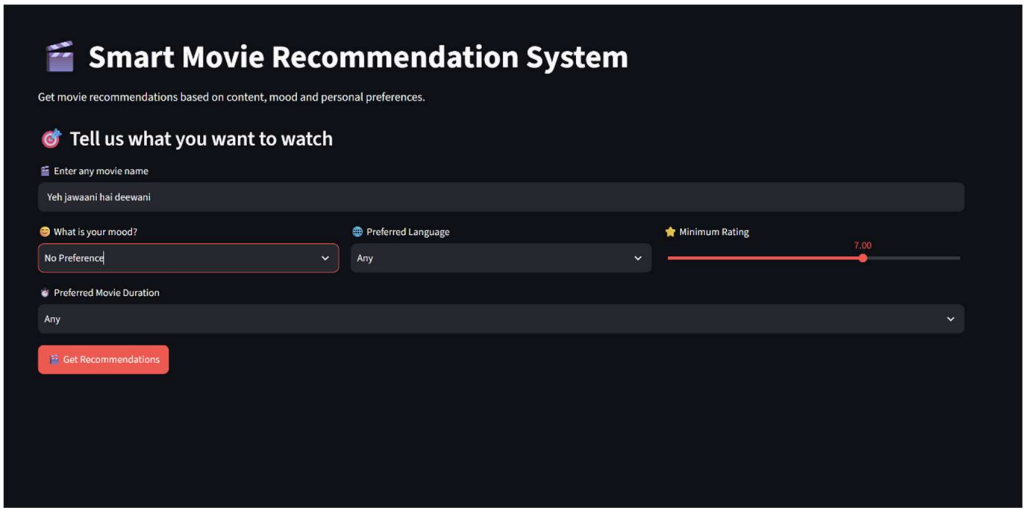


Fig. 9.2: Movie Search and Selection

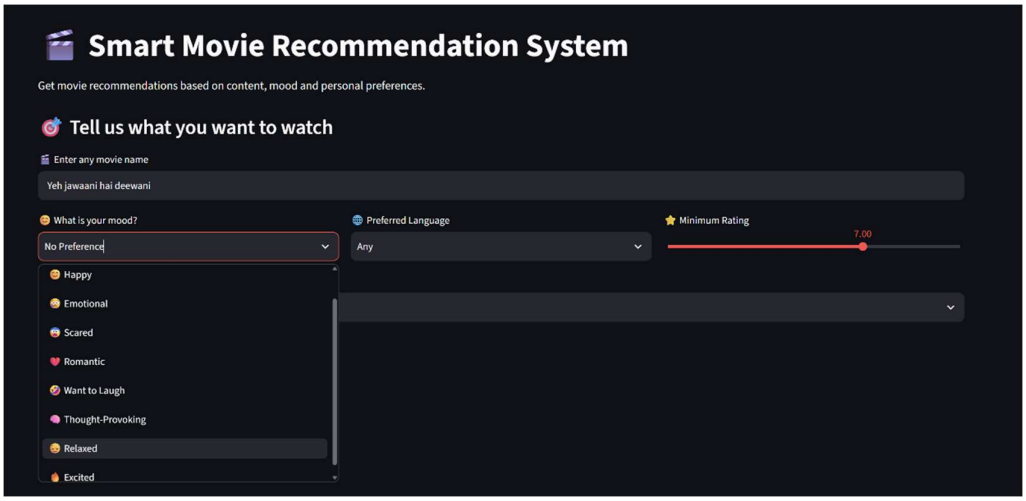


Fig. 9.3: User Preference Selection

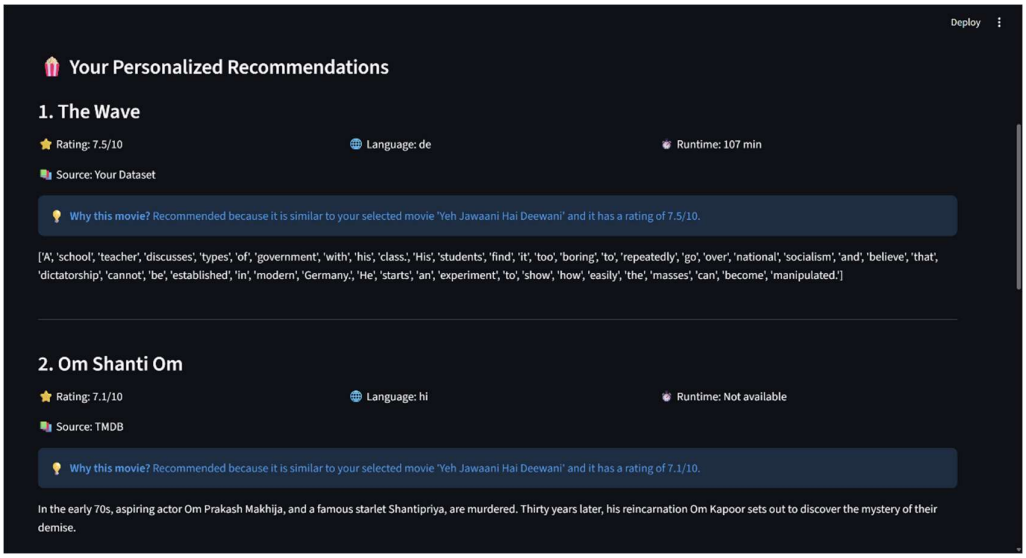


Fig. 9.4: Movie Recommendations

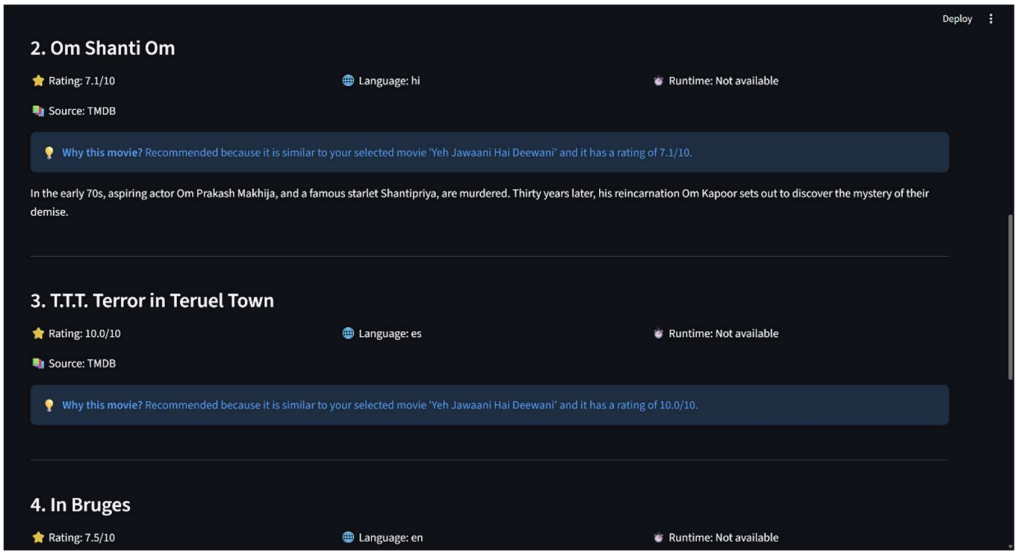
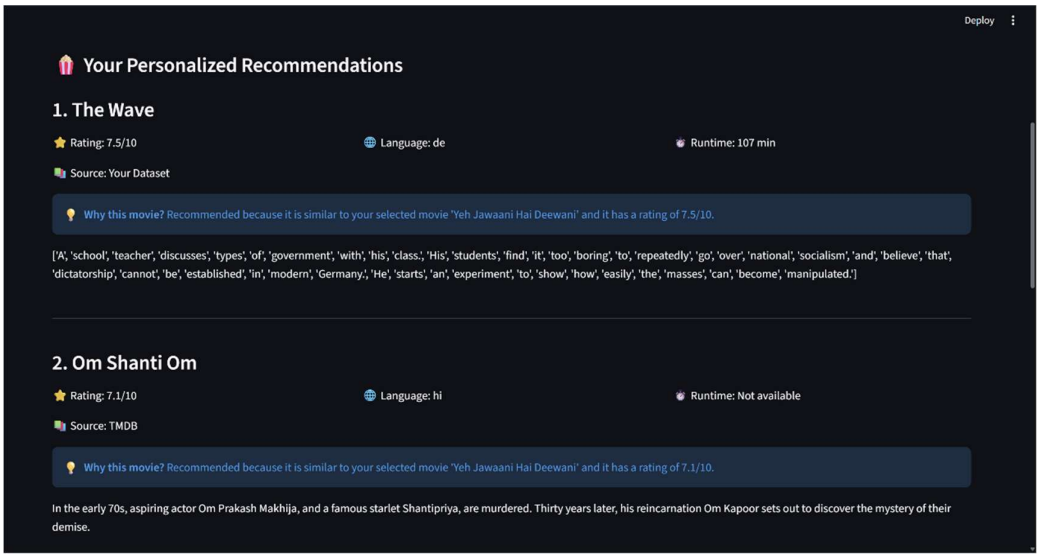
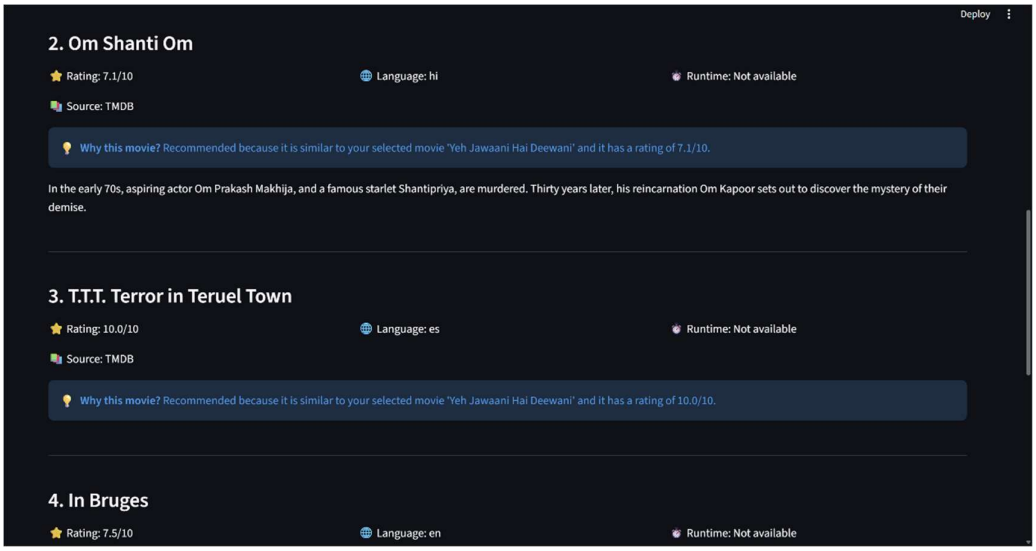


Fig. 9.5: Recommendation Explanations





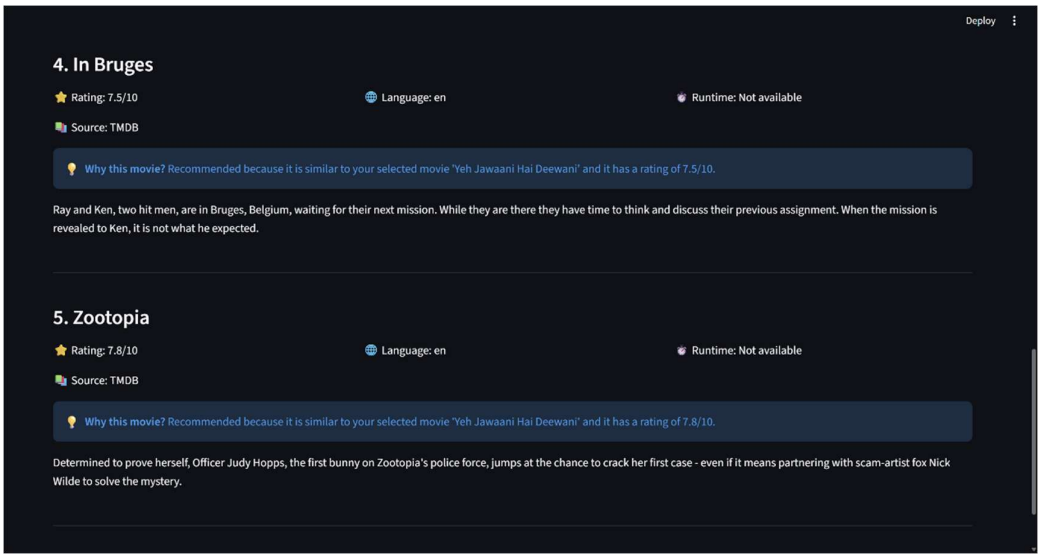


Fig. 9.6: Final Recommendation Results

CHAPTER 10
CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

The Smart Movie Recommendation System was developed to help users discover movies based on their interests and preferences. The system uses content-based filtering with TF-IDF vectorization and cosine similarity to identify movies with similar content. Movie details such as genres, keywords, overview, cast, and director are used to generate recommendations.

The system also integrates the TMDB API to retrieve additional movie suggestions. User preferences, including mood, language, minimum rating, and runtime, are considered during the ranking process. The Streamlit interface displays up to five recommended movies with explanations, making the results easy to understand.

Overall, the project demonstrates how machine learning techniques and API integration can be combined to create a simple and personalized movie recommendation application.

10.2 Future Scope

- Advanced Recommendation Techniques: Integrate semantic embeddings to identify similarities beyond matching words.
- Collaborative Filtering: Include user ratings and viewing history to recommend movies based on the interests of similar users.
- Improved Personalization: Learn from users' feedback to improve future recommendations.
- Multilingual Recommendations: Enhance support for movies across different languages and regional film industries.
- Enhanced User Interface: Add features such as watchlists, movie posters, and detailed movie information.

- Recommendation Evaluation: Use suitable evaluation metrics and user feedback to measure and improve recommendation quality.

- Mobile app and reminders**:** notify students about application deadlines and hackathon registrations.

CHAPTER 11
REFERENCES

[[1] F. Pedregosa, G. Varoquaux, A. Gramfort et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[2] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[3] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

[4] P. Resnick and H. R. Varian, “Recommender Systems,” Communications of the ACM, vol. 40, no. 3, pp. 56–58, 1997.

[5] J. B. Schafer, D. Frankowski, J. L. Herlocker and S. Sen, “Collaborative Filtering Recommender Systems,” in The Adaptive Web, Springer, pp. 291–324, 2007.

[6] Scikit-learn documentation: Feature Extraction and TF-IDF Vectorization. Available: https://scikit-learn.org/stable/modules/feature_extraction.html.

[7] Scikit-learn documentation: Cosine Similarity. Available: https://scikit-learn.org/stable/modules/metrics.html#cosine-similarity.

[8] Python Software Foundation, Python documentation. Available: https://docs.python.org/3/.

[9] Streamlit documentation. Available: https://docs.streamlit.io/.

[10] The Movie Database (TMDB) API documentation. Available: https://developer.themoviedb.org/docs.

[11] Pandas documentation. Available: https://pandas.pydata.org/docs/.

[12] NumPy documentation. Available: https://numpy.org/doc/.

APPENDIX A: API ENDPOINTS

Table A.1: API endpoints used in the Smart Movie Recommendation System

Method	Endpoint	Purpose
GET	/search/movie	Searches for movies based on the title entered by the user.
GET	/movie/{movie_id}/similar	Retrieves similar movies from TMDB for a selected movie.